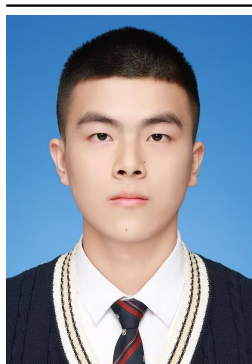


Homework 3

级数求和、机器精度与舍入误差实验报告

姜玥晟

2026-04-21



项目	内容
源题编号	HW03
学生姓名	姜玥晟
报告主题	Gregory-Leibniz 级数、Machin 公式、端序、机器精度与不稳定递推
实验环境	C 数值程序、quadmath 与统一运行日志

目录

I. Gregory-Leibniz 级数计算圆周率	3
II. Machin 公式计算圆周率	7
III. 机器端序检测	10
IV. 机器精度测量	11
V. 不稳定递推与稳定改写	12

I. Gregory-Leibniz 级数计算圆周率

本节利用 Gregory-Leibniz 级数

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

计算 π 。

1.1 Problem 1(1): 有限项计算

相关脚本:

- 本地 scripts/problem1.c
- GitHub scripts/problem1.c

1.1.1 待求问题

取大约 500000 项进行计算。

1.1.2 解决方式

本题的计算过程可表示为如下伪代码:

Input : number of terms N

Output: forward sum, backward sum, and absolute errors

```
sum_forward <- 0
for k from 0 to N - 1 do
    term <- (-1)^k / (2k + 1)
    sum_forward <- sum_forward + term
end for

sum_backward <- 0
for k from N - 1 down to 0 do
    term <- (-1)^k / (2k + 1)
    sum_backward <- sum_backward + term
end for

pi_forward <- 4 * sum_forward
pi_backward <- 4 * sum_backward
compare both values with reference pi
```

程序同时输出 `double` 与 `__float128` 的结果，以区分截断误差和舍入误差的影响。

1.1.3 问题答案

取 $N = 500000$ 项时，`double` 计算结果如下。

方法	double 结果	绝对误差
正序	3.141590653589692	2.000000101088e-06
逆序	3.1415906535897933	1.99999999835e-06

`__float128` 计算结果如下。

方法	__float128 结果	绝对误差
正序	3.14159065358979324046264338326956215	1.99999999998e-06
逆序	3.14159065358979324046264338326950321	1.99999999998e-06

1.1.4 分析

本题的主导误差并不是求和顺序造成的舍入误差，而是级数本身的截断误差。对 Gregory-Leibniz 级数有

$$\left| \pi - 4 \sum_{k=0}^{N-1} \frac{(-1)^k}{2k+1} \right| \leq \frac{4}{2N+1},$$

当 $N=500000$ 时，这一上界本身就处于 10^{-6} 数量级。逆序求和确实略微减轻了舍入误差，但并不能改变 Gregory-Leibniz 级数收敛过慢这一根本事实。

1.2 Problem 1(2): 两种求和顺序比较

相关脚本：

- 本地 `scripts/problem1.c`
- GitHub `scripts/problem1.c`

1.2.1 待求问题

分别采用按正序求和与按逆序求和两种实现。

1.2.2 解决方式

本题的计算过程可表示为如下伪代码：

Input : number of terms N

Output: forward sum, backward sum, and absolute errors

```
sum_forward <- 0
for k from 0 to N - 1 do
    term <- (-1)^k / (2k + 1)
    sum_forward <- sum_forward + term
end for

sum_backward <- 0
for k from N - 1 down to 0 do
    term <- (-1)^k / (2k + 1)
    sum_backward <- sum_backward + term
end for
```

```
pi_forward <- 4 * sum_forward
pi_backward <- 4 * sum_backward
compare both values with reference pi
```

程序同时输出 `double` 与 `__float128` 的结果，以区分截断误差和舍入误差的影响。

1.2.3 问题答案

取 $N = 500000$ 项时，`double` 计算结果如下。

方法	double 结果	绝对误差
正序	3.141590653589692	2.000000101088e-06
逆序	3.1415906535897933	1.999999999835e-06

`__float128` 计算结果如下。

方法	__float128 结果	绝对误差
正序	3.14159065358979324046264338326956215	1.999999999998e-06
逆序	3.14159065358979324046264338326950321	1.999999999998e-06

逆序求和略优于正序求和，但两者的差距非常有限。对 `double` 而言，逆序结果在最后几位上更接近参考值；对 `__float128` 而言，两种顺序在当前展示精度下已几乎不可区分。

1.2.4 分析

本题的主导误差并不是求和顺序造成的舍入误差，而是级数本身的截断误差。对 Gregory-Leibniz 级数有

$$\left| \pi - 4 \sum_{k=0}^{N-1} \frac{(-1)^k}{2k+1} \right| \leq \frac{4}{2N+1},$$

当 $N=500000$ 时，这一上界本身就处于 10^{-6} 数量级。逆序求和确实略微减轻了舍入误差，但并不能改变 Gregory-Leibniz 级数收敛过慢这一根本事实。

1.3 Problem 1(3): 精度判断与原因说明

相关脚本：

- 本地 scripts/problem1.c
- GitHub scripts/problem1.c

1.3.1 待求问题

比较两种实现的精度差异，并说明哪一种更准确以及原因。

1.3.2 解决方式

本题的计算过程可表示为如下伪代码：

Input : number of terms N

Output: forward sum, backward sum, and absolute errors

```
sum_forward <- 0
for k from 0 to N - 1 do
    term <- (-1)^k / (2k + 1)
    sum_forward <- sum_forward + term
end for

sum_backward <- 0
for k from N - 1 down to 0 do
    term <- (-1)^k / (2k + 1)
    sum_backward <- sum_backward + term
end for

pi_forward <- 4 * sum_forward
```

```
pi_backward <- 4 * sum_backward
compare both values with reference pi
```

程序同时输出 double 与 __float128 的结果，以区分截断误差和舍入误差的影响。

1.3.3 问题答案

两种实现都无法达到 10 位小数精度，原因在于 Gregory-Leibniz 级数本身收敛过慢，主导误差来自级数截断而不是舍入顺序。日志还给出了达到 10 位小数大约需要 40000000000 项的估计，说明题目中给出的 500000 项主要用于观察误差机制，而不是为了真正完成 10 位十进制精度。

1.3.4 分析

本题的主导误差并不是求和顺序造成的舍入误差，而是级数本身的截断误差。对 Gregory-Leibniz 级数有

$$\left| \pi - 4 \sum_{k=0}^{N-1} \frac{(-1)^k}{2k+1} \right| \leq \frac{4}{2N+1},$$

当 $N=500000$ 时，这一上界本身就处于 10^{-6} 数量级。逆序求和确实略微减轻了舍入误差，但并不能改变 Gregory-Leibniz 级数收敛过慢这一根本事实。

II. Machin 公式计算圆周率

本节利用 Machin 公式

$$\frac{\pi}{4} = 4 \arctan\left(\frac{1}{5}\right) - \arctan\left(\frac{1}{239}\right)$$

以及反正切级数

$$\arctan(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots$$

2.1 Problem 2(1): 公式构造

相关脚本：

- 本地 scripts/problem2.c
- GitHub scripts/problem2.c

2.1.1 待求问题

利用 Machin 公式与反正切级数构造高精度圆周率计算方法。

2.1.2 问题答案

由 Machin 公式先把圆周率写成

$$\pi = 16 \arctan\left(\frac{1}{5}\right) - 4 \arctan\left(\frac{1}{239}\right).$$

对 $|x| < 1$, 反正切函数的交错级数展开为

$$\arctan(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{2k+1}.$$

于是本小问可构造截断计算式

$$\pi_{m,n} = 16 \sum_{k=0}^m (-1)^k \frac{(1/5)^{2k+1}}{2k+1} - 4 \sum_{k=0}^n (-1)^k \frac{(1/239)^{2k+1}}{2k+1}.$$

因为 $1/5$ 和 $1/239$ 均小于 1, 两组级数项绝对值单调下降。由交错级数余项估计, 截断后总误差满足

$$|\pi - \pi_{m,n}| \leq 16 \frac{(1/5)^{2m+3}}{2m+3} + 4 \frac{(1/239)^{2n+3}}{2n+3}.$$

因此, 给定目标绝对误差 ε 时, 只需选择 m, n 使

$$16 \frac{(1/5)^{2m+3}}{2m+3} + 4 \frac{(1/239)^{2n+3}}{2n+3} < \varepsilon.$$

若使用统一的单项阈值 τ , 可分别累加到下一项满足

$$\frac{(1/5)^{2m+3}}{2m+3} < \tau, \quad \frac{(1/239)^{2n+3}}{2n+3} < \tau.$$

此时总误差不超过 20τ 。本小问到此完成计算方法的构造; 具体数值结果放在下一小问给出。

2.2 Problem 2(2): 精度目标

相关脚本:

- 本地 scripts/problem2.c
- GitHub scripts/problem2.c

2.2.1 待求问题

将 π 计算到 30 位小数。

2.2.2 解决方式

沿用上一问构造的 Machin 截断式。若要求 30 位小数可靠，截断误差应小于半个末位单位，即 5×10^{-31} 。具体计算流程如下：

Input : $x_1 = 1/5$, $x_2 = 1/239$, tolerance $\tau = 1e-35$

Output: π to at least 30 decimal places

```
function atan_series(x, tau):
    sum <- 0
    k <- 0
    repeat
        term <- (-1)^k * x^(2k+1) / (2k+1)
        sum <- sum + term
        k <- k + 1
    until abs(term) < tau
    return sum

a <- atan_series(1/5, tau)
b <- atan_series(1/239, tau)
pi <- 16a - 4b
```

实现使用 `__float128` 与 `quadmath.h`，单项阈值取 10^{-35} 。按上一问的误差界，两个反正切级数的加权截断误差可保守估计为

$$16 \times 10^{-35} + 4 \times 10^{-35} = 2 \times 10^{-34},$$

因此截断误差低于 5×10^{-31} ，不会影响前 30 位小数的判定。

2.2.3 问题答案

最终得到

$$\pi \approx 3.141592653589793238462643383279502797,$$

因此 Machin 公式在本题精度目标下满足 30 位小数要求；程序相对 `acosq(-1)` 的显示差异为 `0.0000000000000000e+00`，截断误差界约为 2×10^{-34} 。

2.2.4 分析

与第一题相比，Machin 公式的优势来自“小参数级数”的快速收敛。由于 $1/5$ 和 $1/239$ 都远小于 1，高阶项会迅速衰减，因此所需项数极少即可达到高精度目标。这里的精度判断同时依赖数值输出和余项界：输出验证实现结果，余项界解释为什么该项数足以支撑 30 位小数。

III. 机器端序检测

3.1 Problem 3: 机器端序检测

相关脚本：

- 本地 scripts/problem3.c
- GitHub scripts/problem3.c

3.1.1 待求问题

编写 C 程序，判断当前计算机采用 big-endian 还是 little-endian 存储。

3.1.2 解决方式

判断过程可概括为：

Input : probe integer 0x01020304

Output: machine byte order

```
store integer in memory
view its address as byte array
if first byte == 0x04 then
    machine is little-endian
else if first byte == 0x01 then
    machine is big-endian
end if
```

3.1.3 问题答案

本机探针字节顺序为 04 03 02 01，因此当前机器采用 little-endian 存储。

3.1.4 分析

端序只描述多字节数据在内存中的排布方式，并不等同于处理器的全部体系结构特征。通过字节探针判断端序，是因为该方法直接考察同一整数在内存中的最低地址字节内容，因此具有明确而稳定的解释。

IV. 机器精度测量

4.1 Problem 4: 机器精度测量

相关脚本:

- 本地 scripts/problem4.c
- GitHub scripts/problem4.c

4.1.1 待求问题

编写程序, 分别测量当前系统 float、double 与 quadruple precision 的 machine epsilon。

4.1.2 解决方式

设浮点格式采用二进制、规格化有效位数为 p , 则 1 附近的相邻可表示数为

$$1, \quad 1 + 2^{1-p}, \quad 1 + 2 \cdot 2^{1-p}, \dots$$

因此 1 到下一个大于 1 的浮点数之间的间距为

$$\Delta = 2^{1-p}.$$

machine epsilon 按通常定义就是使浮点运算中 $1 + \varepsilon$ 能够区别于 1 的最小正间距, 即 $\varepsilon = \Delta$ 。在默认 round-to-nearest 模式下, 若增量小于半个间距, 则 $1 + \delta$ 会舍入回 1; 当增量达到一个完整间距 Δ 时, 结果才稳定变为下一个可表示数。

程序从 $\varepsilon = 1$ 开始反复二分, 并检测

$$\text{fl}\left(1 + \frac{\varepsilon}{2}\right) > 1$$

是否仍成立。若成立, 说明当前 $\varepsilon/2$ 仍能被机器区分, 于是继续缩小 ε ; 当该判断第一次失败时, $\varepsilon/2$ 已不能改变 1, 而当前 ε 正好是仍可区分的最小二进制间距。因此算法返回的就是该类型的 machine epsilon。

具体算法如下:

Input : floating-point type T

Output: machine epsilon of T

```
eps <- 1
while 1 + eps / 2 > 1 do
  eps <- eps / 2
```

```
end while
return eps
```

上述过程分别对 float、double 和 __float128 执行。

4.1.3 问题答案

测得结果如下。

类型	machine epsilon
float	1.192092896e-07
double	2.220446049e-16
__float128	1.925929944e-34

三者都与理论参考值完全一致。

4.1.4 分析

对二进制浮点数，machine epsilon 满足

$$\varepsilon = 2^{1-p},$$

其中 p 是有效位数。float、double 与 __float128 的有效位数分别为 24、53、113，因此理论值恰好对应 2^{-23} 、 2^{-52} 与 2^{-112} 。本题结果说明逐次二分法能够正确探测浮点格式的最小分辨能力。

V. 不稳定递推与稳定改写

本节考虑序列

$$z_2 = 2, \quad z_{n+1} = 2^{n-\frac{1}{2}} \sqrt{1 - \sqrt{1 - 4^{1-n} z_n^2}}, \quad n = 2, 3, \dots$$

5.1 Problem 5(1): 数值极限比较

相关脚本：

- 本地 scripts/problem5.c
- GitHub scripts/problem5.c

5.1.1 待求问题

比较数值递推结果与极限 π 。

5.1.2 解决方式

程序同时实现题面中的直接递推与经有理化后的稳定递推:

Input : starting value $z_2 = 2$ and index range n

Output: direct recurrence, stabilized recurrence, and errors

```
for n from 2 upward do
  a <- 4^(1-n) * z_n^2
  z_direct[n+1] <- 2^(n-1/2) * sqrt(1 - sqrt(1 - a))
  z_stable[n+1] <- z_n * sqrt(2 / (1 + sqrt(1 - a)))
  compare both values with pi
end for
```

稳定形式利用了恒等变换

$$1 - \sqrt{1 - a} = \frac{a}{1 + \sqrt{1 - a}},$$

从而避免直接计算两个接近量之差。

5.1.3 问题答案

代表性结果如下。

(n)	直接值	直接误差	稳定值	稳定误差
20	3.141594125195	1.472e-06	3.141592653571	-1.880e-11
25	3.142451272494	8.586e-04	3.141592653590	-1.865e-14
30	4.000000000000	8.584e-01	3.141592653590	-4.441e-16
31	8.000000000000	4.858e+00	3.141592653590	-4.441e-16
40	4096.000000000000	4.093e+03	3.141592653590	-4.441e-16

因此，直接递推在 $n=30$ 左右开始彻底失稳，而稳定改写后仍保持对 π 的收敛。

5.1.4 分析

失稳的根源在于表达式

$$1 - \sqrt{1 - a_n}.$$

当 a_n 很小时， $\sqrt{1 - a_n}$ 与 1 极其接近，二者相减会造成严重相消误差；随后外层的放大因子 $2^{n-\frac{1}{2}}$ 又会把这部分误差迅速放大，从而使递推结果偏离 π 。稳定改写并没有改变问题的数学极限，而是改变了误差传

播路径，因此能够显著抑制 rounding error propagation。

5.2 Problem 5(2): 不稳定原因解释

相关脚本：

- 本地 scripts/problem5.c
- GitHub scripts/problem5.c

5.2.1 待求问题

解释为何当 $n \geq 30$ 时会出现 rounding error propagation。

5.2.2 解决方式

程序同时实现题面中的直接递推与经有理化后的稳定递推：

Input : starting value $z_2 = 2$ and index range n

Output: direct recurrence, stabilized recurrence, and errors

```
for n from 2 upward do
  a <- 4^(1-n) * z_n^2
  z_direct[n+1] <- 2^(n-1/2) * sqrt(1 - sqrt(1 - a))
  z_stable[n+1] <- z_n * sqrt(2 / (1 + sqrt(1 - a)))
  compare both values with pi
end for
```

稳定形式利用了恒等变换

$$1 - \sqrt{1 - a} = \frac{a}{1 + \sqrt{1 - a}},$$

从而避免直接计算两个接近量之差。

5.2.3 问题答案

记

$$a_n = 4^{1-n} z_n^2, \quad d_n = 1 - \sqrt{1 - a_n}.$$

沿直接递推的双精度数值路径考察 d_n 的计算误差，可得到下表。

更新步	a_n	精确 d_n	双精度 d_n	d_n 相对误差
28 $\rightarrow$ 29	5.551×10^{-16}	2.776×10^{-16}	3.331×10^{-16}	20.0%
29 $\rightarrow$ 30	1.665×10^{-16}	8.327×10^{-17}	1.110×10^{-16}	33.3%
30 $\rightarrow$ 31	5.551×10^{-17}	2.776×10^{-17}	1.110×10^{-16}	300.0%
31 $\rightarrow$ 32	5.551×10^{-17}	2.776×10^{-17}	1.110×10^{-16}	300.0%

这些差值误差再进入

$$z_{n+1} = 2^{n-\frac{1}{2}} \sqrt{d_n},$$

其外层放大因子和最终偏差如下。

更新步	$2^{n-\frac{1}{2}}$	直接 z_{n+1}	$ z_{n+1} - \pi $
28 $\rightarrow$ 29	1.898×10^8	3.464101615138	3.225×10^{-1}
29 $\rightarrow$ 30	3.796×10^8	4.000000000000	8.584×10^{-1}
30 $\rightarrow$ 31	7.593×10^8	8.000000000000	4.858
31 $\rightarrow$ 32	1.519×10^9	16.000000000000	12.858

因此, $n \geq 30$ 附近的失稳不是单纯的趋势判断, 而是因为直接差值 d_n 已进入 $10^{-17} \sim 10^{-16}$ 量级; 双精度舍入使 d_n 的相对误差达到 33.3% 到 300%, 经平方根和 $10^8 \sim 10^9$ 量级的外层因子后, 直接结果从 π 附近跳到 4, 8, 16, ...。

5.2.4 分析

在 binary64 中, 1 下方相邻浮点间距为 $2^{-53} \approx 1.110 \times 10^{-16}$ 。当 $n = 29 \rightarrow 30$ 时, 精确差值 $d_n \approx 8.327 \times 10^{-17}$, 已经小于这一间距; 当 $n = 30 \rightarrow 31$ 时, 精确差值进一步降到 2.776×10^{-17} , 但直接相减得到的双精度差值仍为 1.110×10^{-16} , 相当于把 d_n 放大了约 4 倍。由于 z_{n+1} 与 $\sqrt{d_n}$ 成正比, 这一 4 倍差值误差会变成约 2 倍的 z 值误差, 正对应 $4 \rightarrow 8 \rightarrow 16$ 的后续发散。稳定改写

$$d_n = \frac{a_n}{1 + \sqrt{1 - a_n}}$$

避免了两个接近 1 的数直接相减, 所以在 $n = 30$ 时稳定递推仍给出 3.141592653590, 误差仅约 4.441×10^{-16} 。